Name: Javaughn Stevens

CMSC 204 Dictionary Builder Project 4 Pseudocode

// pseudocode for DictionaryBuilder.java

Declare a public class DictionaryBuilder

Declare a private integer variable estimatedEntries
 Declare a private String variable filename
 Declare a private integer variable totalWordCount initialized to 0
 Declare a private integer variable size initialized to 0
 Declare a private MyLinkedList array linkedHashTable
 Declare a public static final double loadFactor initialized to 0.6

Declare a public constructor DictionaryBuilder with integer parameter estimatedEntries
 Set this.estimatedEntries equal to estimatedEntries
 Call createHashTable with estimatedEntries
 // end of constructor

Declare a public constructor DictionaryBuilder with String parameter filename that throws FileNotFoundException
Set this.filename equal to filename
Declare a File variable myFile equal to new File(filename)
Declare integer estimatedEntries equal to (myFile.length divided by 100)
Call function createHashTable with estimatedEntries
Declare a Scanner variable dictionaryScanner equal to new Scanner(myFile)

While dictionaryScanner.hasNext() equals true
 Call addWord with dictionaryScanner.next()
 // end of while loop

Close the scanner object
// end of constructor

Declare a public void method addWord with String parameter word
Set word equal to simplifyWord(word)

If word.length equals 0
End function
// end if

Declare a integer wordIndex equal to hasher(word)
Declare a MyLinkedList variable linkedBucket equal to linkedHashTable[wordIndex]

Declare a DictionaryEntry variable userEntry equal to linkedBucket.findNode(word)

If userEntry equals null
Call linkedBucket.insertNode(word)
Increment size by 1
Else
Increment userEntry.count by 1
// end if

Increment totalWordCount by 1
// end of addWord method

Declare a public void method removeWord with String parameter word that throws DictionaryEntryNotFoundException

Set word equal to simplifyWord(word)
Declare a integer wordIndex equal to hasher(word)
Declare a DictionaryEntry variable userEntry equal to linkedHashTable[wordIndex].findNode(word)

If userEntry equals null
 Throw DictionaryEntryNotFoundException with message "The word was not found."
 // end if

Set totalWordCount equal to totalWordCount minus userEntry.count
Decrement size by 1

Call linkedHashTable[wordIndex].deleteNode(word)
 // end of removeWord method

Declare a public integer method getFrequency with String parameter word

Set word equal to simplifyWord(word)

If word.length equals 0
Return 0
 // end if

Declare a integer wordIndex equal to hasher(word)
Declare a DictionaryEntry variable userEntry equal to linkedHashTable[wordIndex].findNode(word)

 If userEntry equals null
 Return 0
 Else
 Return userEntry.count
 // end if

// end of getFrequency method

Declare a public List<String> method getAllWords

Declare a List<String> variable wordList equal to new ArrayList

While i equals 0 to linkedHashTable.length - 1
Call linkedHashTable[i].collectWords(wordList)

Increment i by 1

// end for loop

Call function Collections.sort(wordList)

Return wordList
// end of getAllWords method

Declare a private void method createHashTable with integer parameter estimatedEntries

Declare integer tableSize equal to next4kPlus3Prime(estimatedEntries divided by loadFactor)

Set linkedHashTable equal to new MyLinkedList array of size tableSize

While i equals 0 to tableSize - 1
Set linkedHashTable[i] equal to new MyLinkedList
Increment i by 1
// end for loop

// end of createHashTable method

Declare a private String method simplifyWord with String parameter word

Set word to all lowercase characters

Declare StringBuilder variable simplified equal to new StringBuilder

```
While i equals 0 to word.length - 1
 Declare char variable character equal to word.charAt(i)

If character is between 'a' and 'z' inclusive
 Call simplified.append(character)

Increment i by 1
 // end if

// end for loop

Return simplified.toString()
 // end of simplifyWord method

Declare a private integer method hasher with String parameter
word

Declare integer hash initialized to 0

While i equals 0 to word.length - 1
Set hash equal to 31 * hash + word.charAt(i)
Increment i by 1
 // end for loop

Declare integer finalHashCode

 If hash is less than 0
 Set finalHashCode equal to hash multiplied by -1
 Else
 Set finalHashCode equal to hash
 // end if

Return the modulus of finalHashCode and linkedHashTable.length
 // end of hasher method
```

Declare a private integer method next4kPlus3Prime with integer parameter value

Declare boolean variable continuous equal to true

While continuous equals true

If isPrime(value) equals true AND value modulo 4 equals 3
Return value
Else
Increment value by 1
// end if

// end while loop

Return value
// end of method

Declare a private boolean method isPrime with integer parameter n (given)

If n is less than or equal to 1
Return false

If n equals 2 OR n equals 3
Return true

If n modulo 2 equals 0 OR n modulo 3 equals 0
Return false

// end of if

While i equals 5 to square root of n increment by 6
If n modulo i equals 0 OR n modulo (i + 2) equals 0
Return false

```
Increment i by 1
 // end for loop

 Return true
 // end of isPrime method

 Declare a public integer method getTotalWordCount
 Return totalWordCount
 // end of method

 Declare a public integer method getUniqueWordCount
 Return size
 // end of method

Declare a public double method getLoadFactor
Return size divided by linkedHashTable.length as double
// end of method

// end of class DictionaryBuilder.java and DictionaryBuilder.java
pseudocode

// pseudocode for DictionaryEntry.java

Declare a public class DictionaryEntry
Declare a public String variable word
Declare a public integer variable count

Declare a public constructor DictionaryEntry with String
parameter word

Set this.word equal to word
Set this.count equal to 1

// end of constructor

// end of class DictionaryEntry and DictionaryEntry.java
pseudocode
```

```
// pseudocode for DictionaryShell.java

Declare a public class DictionaryShell

Declare a public constructor DictionaryShell with no parameter arguments

// end of constructor

Declare a public static void method main with String array parameter args
Declare Scanner variable userInput for System.in
Declare DictionaryBuilder variable userDictionary initialized to null

Try
If args.length is greater than 0
 Set userDictionary equal to new DictionaryBuilder using args[0]
 Else
 Set userDictionary equal to new DictionaryBuilder using 100
 // end if

Catch FileNotFoundException
 Display "File not found."
 Call userInput.close()
 End main function

Catch Exception
 Display exception.getMessage()
 Call userInput.close()
 End main function

Display "Welcome to the Dictionary Builder CLI."
Display "Available commands: add <word>, delete <word>, search <word>, list, stats, exit >"

Declare boolean variable continuous equal to true
```

```
While continuous equals true

Display "> "

Declare String variable line equal to userInput.nextLine().trim()

 If line.length equals 0
 Continue
 // end if

Declare Scanner variable lineScanner equal to new Scanner(line)
Declare String variable command equal to
lineScanner.next().toLowerCase()
Declare String variable word initialized to null

If lineScanner.hasNext() equals true
Set word equal to lineScanner.next()
// end if

If command equals "add"

If word equals null
Display "Usage: add <word>"
Continue
 // end if

Call userDictionary.addWord(word)
Display """ + word.toLowerCase() + "" added."

Else if command equals "delete"

 If word equals null
 Display "Usage: delete <word>"
 Continue
 // end if
```

```
Try
 Call userDictionary.removeWord(word)
 Display """ + word.toLowerCase() + "" deleted."

Catch DictionaryEntryNotFoundException
 Display exception.getMessage()

Else if command equals "search"

If word equals null
 Print "Usage: search <word>"
 Continue
 // end if

Declare integer frequency equal to
userDictionary.getFrequency(word)

If frequency is greater than 0
 Print frequency + " instance(s) of "" + word.toLowerCase() + ""
found."
 Else
 Print """ + word.toLowerCase() + "" not found."
 // end if

Else if command equals "list"

Declare List<String> variable dictionaryWords equal to
userDictionary.getAllWords()

While i equals 0 to dictionaryWords.size() - 1
Print dictionaryWords.get(i)
Increment i by 1
 // end for loop

Else if command equals "stats"
```

```
Display "Total words: " + userDictionary.getTotalWordCount()
Display "Total unique words: " +
userDictionary.getUniqueWordCount()
Display "Estimated load factor: %.2f" using
userDictionary.getLoadFactor()

Else if command equals "exit"

 Print "Quitting..."
 Call lineScanner.close()
 Return

Else

Print "Command is invalid."

Close lineScanner scanner

// end of while loop

Close userInput scanner

// end of main method

// end of class DictionaryShell and DictionaryShell.java
pseudocode
```

```
// pseudocode for MyLinkedList.java

Declare a public class MyLinkedList
Declare a DictionaryEntry variable data
Declare a Node variable nextNode
```

Declare a constructor Node with DictionaryEntry parameter data

Set this.data equal to data
 Set this.nextNode equal to null

// end of constructor

// end of Node class

Declare a private Node variable headNode

Declare a public DictionaryEntry method findNode with String parameter word

Declare a Node variable currentNode equal to headNode

While currentNode is not null

 If currentNode.data.word equals word
 Return currentNode.data
 // end if

Set currentNode equal to currentNode.nextNode
// end while loop

Return null

// end of findNode method

Declare a public void method insertNode with String parameter word

Declare Node variable newNode equal to new Node containing new DictionaryEntry(word)

Set newNode.nextNode equal to headNode
Set headNode equal to newNode

// end of insertNode method

Declare a public void method deleteNode with String parameter word that throws DictionaryEntryNotFoundException

Declare a Node variable currentNode equal to headNode
Declare a Node variable previousNode equal to null

While currentNode is not null

If currentNode.data.word equals word

If previousNode equals null
 Set headNode equal to currentNode.nextNode
 Else
 Set previousNode.nextNode equal to currentNode.nextNode
 // end if

Return

// end if

Set previousNode equal to currentNode
 Set currentNode equal to currentNode.nextNode

// end while loop

Throw DictionaryEntryNotFoundException with message "The word was not found in the dictionary."

// end of deleteNode method

Declare a public List<String> method getWords

Declare a List<String> variable wordList equal to new ArrayList

Declare a Node variable currentNode equal to headNode

While currentNode is not null

Call wordList.add(currentNode.data.word)

Set currentNode equal to currentNode.nextNode

// end while loop

Return wordList

// end of getWords method

Declare a public void method collectWords with List<String> parameter wordList

Declare Node variable currentNode equal to headNode

While currentNode is not null

Call wordList.add(currentNode.data.word)

Set currentNode equal to currentNode.nextNode

// end while loop

// end of collectWords method

// end of class MyLinkedList and MyLinkedList.java pseudocode

// pseudocode for DictionaryEntryNotFoundException.java

Declare a public class DictionaryEntryNotFoundException that extends Exception

Declare a public constructor DictionaryEntryNotFoundException with String parameter message
Call superclass constructor Exception with message
// end of constructor

// end of class DictionaryEntryNotFoundException and DictionaryEntryNotFoundException.java pseudocode